

Uniform noise বুঝতে হলে আগে “uniform” শব্দটার মানে ধরতে হবে—সব জায়গায় সমানভাবে ছড়ানো। এখন পুরো বিষয়টা ধাপে ধাপে সহজ কিন্তু গভীরভাবে বুঝাই।

◆ Uniform Noise কী?

Uniform noise হলো এমন এক ধরনের noise যেখানে নির্দিষ্ট একটা range-এর মধ্যে সব মান সমান probability নিয়ে আসে।

☞ মানে:

- Noise যদি $[a, b]$ range-এর মধ্যে থাকে
- তাহলে ওই range-এর যেকোনো মান আসার chance একই

◆ Mathematical Representation

Uniform noise-এর probability density function (PDF):

$$f(x) = \begin{cases} \frac{1}{b-a}, & a \leq x \leq b \\ 0, & \text{otherwise} \end{cases}$$

✦ এখানে:

- (a) = minimum value
- (b) = maximum value

☞ অর্থ:

- (a) থেকে (b) এর মধ্যে সব value সমান সম্ভাবনায় আসবে
- এর বাইরে কিছুই আসবে না

◆ Intuition (সহজভাবে বুঝি)

ধরো তুমি একটা random number generate করছো:

☞ Range: -2 থেকে +2

তাহলে:

- -1.5 আসার chance = 0.5 আসার chance = 1.8 আসার chance
✓□ সব একই

◆ Graph কেমন হয়?

- Graph হবে একটা **flat rectangle**
- Height constant থাকে
- কোন peak নেই (Gaussian এর মতো না)

☞ তাই একে বলে **Rectangular Distribution**

◆ Image Processing-এ Uniform Noise

যখন image-এ uniform noise যোগ হয়:

☞ Effect:

- Pixel value randomভাবে বাড়ে বা কমে
- কিন্তু change সব জায়গায় **equally random**

☞ Formula:

$$\begin{bmatrix} g(x,y) = f(x,y) + n(x,y) \end{bmatrix}$$

- $(f(x,y))$ = original image
- $(n(x,y))$ = uniform noise
- $(g(x,y))$ = noisy image

◆ Characteristics (মুখ্য বৈশিষ্ট্য)

- ✓ ☐ নির্দিষ্ট range থাকে
- ✓ ☐ সব value equally likely
- ✓ ☐ Gaussian noise-এর মতো smooth না
- ✓ ☐ sudden spike বা sharp variation হতে পারে

◆ Mean ও Variance

Uniform noise-এর:

- Mean:

$$\mu = \frac{a + b}{2}$$

- Variance:

$$\sigma^2 = \frac{(b - a)^2}{12}$$

👉 এইগুলো exam-এ খুব important !

◆ Real-Life Example

- Random sensor error (bounded)
- Analog-to-digital conversion error
- Quantization error (অনেক সময় uniform ধরা হয়)

◆ Uniform vs Gaussian Noise (Short Compare)

Feature	Uniform Noise	Gaussian Noise
Distribution	Flat	Bell-shaped
Probability	Equal	Center বেশি
Range	Fixed	Infinite (theoretical)
Smoothness	কম	বেশি

◆ Important Insight (Exam Trick ⚡)

☞ যদি question বলে:

- “equal probability within a range” → Uniform Noise
- “mean centered, bell curve” → Gaussian Noise

Code:1

◆ 1. Library Import

```
import numpy as np
import matplotlib.pyplot as plt
```

☞ numpy (np)

- Mathematical calculation, array handling, random number generate করার জন্য

☞ matplotlib.pyplot (plt)

- Graph/plot আঁকার জন্য

◆ 2. Signal Generate করা

```
t = np.linspace(0, 1, 1000)
```

☞ 0 থেকে 1 পর্যন্ত 1000টা point তৈরি করছে

→ □ এটা time axis (t)

```
signal = np.sin(2 * np.pi * 5 * t)
```

☞ এখানে sine wave তৈরি করা হচ্ছে

- $2\pi f t$ → standard sine formula
- $f = 5$ Hz (frequency)

→ □ Result:

✓ smooth wave (original clean signal)

◆ 3. Gaussian Noise (AWGN)

```
gaussian_noise = np.random.normal(0, 0.3, len(signal))
```

☞ Gaussian (normal) noise generate করছে

- $0 \rightarrow \text{mean } (\mu)$
- $0.3 \rightarrow \text{standard deviation } (\sigma)$
- $\text{len}(\text{signal}) \rightarrow \text{signal-এর সমান length}$

→ □ random noise values

```
signal_gaussian = signal + gaussian_noise
```

☞ original signal + noise

→ □ noisy signal তৈরি

◆ 4. Uniform Noise

```
uniform_noise = np.random.uniform(-0.3, 0.3, len(signal))
```

☞ uniform distribution থেকে noise নিচ্ছে

- range: -0.3 থেকে +0.3
→ □ সব value equal probability

```
signal_uniform = signal + uniform_noise
```

☞ signal-এর সাথে uniform noise যোগ

◆ 5. Impulse Noise (Salt & Pepper)

```
impulse_noise = np.zeros(len(signal))
```

☞ প্রথমে সব জায়গায় 0 noise

```
num_impulses = int(0.05 * len(signal))
```

☞ 5% sample-এ noise দিবে

→ □ total sample-এর 5%

```
indices = np.random.randint(0, len(signal), num_impulses)
```

☞ random position select করছে

→ □ কোন জায়গায় impulse noise বসবে

```
impulse_noise[indices] = np.random.choice([-1, 1], num_impulses)
```

☞ selected জায়গায়:

- -1 অথবা +1 বসাবে

→ □ sudden spikes তৈরি (salt & pepper effect)

```
signal_impulse = signal + impulse_noise
```

☞ final impulse noisy signal

◆ 6. Plot Setup

```
plt.figure(figsize=(12, 8))
```

☞ পুরো figure size set করছে

◆ 7. Original Signal Plot

```
plt.subplot(4, 1, 1)
plt.plot(t, signal)
plt.title("Original Signal")
```

☞ 4টা plot-এর মধ্যে 1st plot
→ □ clean signal দেখাচ্ছে

◆ 8. Gaussian Noise Plot

```
plt.subplot(4, 1, 2)
plt.plot(t, signal_gaussian)
plt.title("Signal with Gaussian Noise")
```

☞ Gaussian noisy signal

◆ 9. Uniform Noise Plot

```
plt.subplot(4, 1, 3)
plt.plot(t, signal_uniform)
plt.title("Signal with Uniform Noise")
```

☞ uniform noisy signal

◆ 10. Impulse Noise Plot

```
plt.subplot(4, 1, 4)
plt.plot(t, signal_impulse)
plt.title("Signal with Impulse Noise")
```

☞ impulse noisy signal (spikes দেখা যাবে)

◆ 11. Layout + Show

```
plt.tight_layout()
```

☞ plot overlap যেন না হয়, spacing adjust করে

```
plt.show()
```

☞ সব graph display করে

🔥 Final Understanding

☞ এই কোডে তুমি দেখালে:

Noise Type	Effect
Gaussian	smooth random variation
Uniform	bounded random variation
Impulse	sudden spikes

◆ Viva Tip 🔥

☞ যদি examiner জিজ্ঞেস করে:

- Which noise is most realistic?
→ ☐ Gaussian noise
- Which noise causes spikes?
→ ☐ Impulse noise

Code-2:

এই কোডটা মূলত Gaussian noise-এর mean (μ) এবং variance (σ^2) পরিবর্তন করলে distribution কেমন বদলায়—তা histogram দিয়ে দেখায়। এখন একদম line-by-line বুঝি 📖

◆ 1. Library Import

```
import numpy as np
import matplotlib.pyplot as plt
```

🔗 numpy (np) → random number generate, math কাজ

🔗 matplotlib.pyplot (plt) → graph (histogram) আঁকার জন্য

◆ 2. Sample সংখ্যা

```
n = 1000
```

🔗 মোট 1000টা random noise sample তৈরি করবে

→ □ যত বেশি sample, histogram তত smooth হবে

◆ 3. Mean ও Variance Define

```
means = [0, 0, 2]
variances = [0.5, 1, 2]
```

🔗 এখানে ৩টা case বানানো হয়েছে:

Case Mean (μ) Variance (σ^2)

1 0 0.5

2 0 1

3 2 2

→ □ লক্ষ্য:

- প্রথম দুইটা → mean same, variance different
- শেষটা → mean change + variance change

◆ 4. Figure Setup

```
plt.figure(figsize=(10, 6))
```

☞ পুরো plot window-এর size সেট করছে

◆ 5. Loop (৩টা case plot করার জন্য)

```
for i in range(3):
```

☞ loop ৩ বার চলবে (i = 0, 1, 2)

→ □ প্রতিবার আলাদা mean/variance use হবে

◆ 6. Gaussian Noise Generate

```
noise = np.random.normal(means[i], np.sqrt(variances[i]), n)
```

☞ এখানে Gaussian (normal) distribution থেকে data নিচ্ছে

- `means[i]` → mean (μ)
- `np.sqrt(variances[i])` → standard deviation (σ)
☞ কারণ function σ নেয়, σ^2 না
- `n` → total sample (1000)

→ □ Result: random noise data

◆ 7. Subplot Select

```
plt.subplot(3, 1, i+1)
```

☞ 3 row, 1 column layout

→ □ প্রতিবার নতুন subplot তৈরি

- $i=0 \rightarrow$ 1st plot
- $i=1 \rightarrow$ 2nd plot
- $i=2 \rightarrow$ 3rd plot

◆ 8. Histogram Plot

```
plt.hist(noise, bins=30)
```

☞ noise data-এর histogram আঁকছে

- $\text{bins}=30 \rightarrow$ data কে 30টা ভাগে ভাগ করে
→ □ distribution shape দেখা যায় (bell curve)

◆ 9. Title Set

```
plt.title(f"Mean = {means[i]}, Variance = {variances[i]}")
```

☞ প্রতিটা plot-এর উপরে label দেখাবে

→ □ কোন mean/variance use হয়েছে

◆ 10. Layout Adjust

```
plt.tight_layout()
```

☞ subplot গুলোর মধ্যে spacing ঠিক করে

→ □ overlap এড়ায়

◆ 11. Show Plot

`plt.show()`

☞ সব graph display করে

🔥 Conceptual Understanding

☞ এই কোডে তুমি observe করবে:

✓☐ Case 1 vs Case 2 (μ same, σ^2 different)

- Variance $\uparrow \rightarrow$ graph **wider**
 - Variance $\downarrow \rightarrow$ graph **narrow**
-

✓☐ Case 3 (μ change)

- Mean $\uparrow \rightarrow$ graph **ডানদিকে shift**
 - Mean $\downarrow \rightarrow$ graph **বামদিকে shift**
-

◆ Final Insight (VERY IMPORTANT)

☞ Mean (μ):

- শুধু position shift করে

☞ Variance (σ^2):

- spread/width change করে
-

◆ Viva Ready Answer 🔥

☞ Mean controls the central position of the distribution, while variance controls the spread. Increasing variance makes the distribution wider, and changing the mean shifts the distribution left or right without changing its shape.

Code-3:

◆ 1. Input নেওয়া

```
data = input("Enter binary data: ")
```

☞ user keyboard থেকে data দেবে

→ □ এটা string হিসেবে store হবে (যেমন: "10101")

◆ 2. Valid binary check

```
if all(bit in '01' for bit in data):
```

☞ এই লাইনটাই সবচেয়ে important 🍷

কী হচ্ছে এখানে?

- for bit in data
☞ data-এর প্রতিটি character এক এক করে নিচ্ছে
- bit in '01'
☞ check করছে character টা কি '0' বা '1' কিনা
- all(...)
☞ সবগুলো character valid হলে True দিবে

✓ □ Example

☞ যদি input হয়:

- "10101" → সব valid → True
- "10201" → 2 আছে → False

◆ 3. Valid হলে কী হবে

```
received = data
```

☞ transmitted data-টাই received data হিসেবে নিচ্ছে
→ □ এখানে কোনো noise/addition নেই (ideal case)

```
print("Transmitted Data :", data)
print("Received Data      :", received)
```

☞ output দেখাবে:

```
Transmitted Data : 10101
Received Data      : 10101
```

→ □ মানে:
✓ perfect transmission (no error)

◆ 4. Invalid হলে

```
else:
    print("Invalid input! Enter only 0 and 1.")
```

☞ যদি data-তে 0/1 ছাড়া কিছু থাকে
→ □ error message দেখাবে

🔥 Conceptual Understanding

☞ এই কোডে আসলে তুমি দেখাচ্ছে:

- ✓ Data validation
- ✓ Binary input checking
- ✓ Ideal communication (no noise)

◆ Communication Theory Connection

☞ এখানে:

- `data` = transmitted signal
- `received` = received signal

→ □ যেহেতু:

- কোনো noise নাই

☞ তাই:

✓ **Transmitted = Received**

◆ Viva Tip 🔥

☞ যদি examiner জিজ্ঞেস করে:

Q: What is this program doing?

☞ Answer:

This program takes binary input from the user, checks whether it contains only valid bits (0 and 1), and then simulates an ideal transmission where the received data is identical to the transmitted data.

Code-4:

এই কোডটা মূলত **Binary Symmetric Channel (BSC)** এর **entropy** এবং **channel capacity** বের করার জন্য। এখন একদম line-by-line বুঝি ☺

◆ 1. Library Import

```
import math
```

☞ এখানে `math` library ব্যবহার করা হচ্ছে
 → □ কারণ $\log_2$ (log base 2) লাগবে

◆ 2. Input নেওয়া

```
p = float(input("Enter error probability (0 ≤ p ≤ 1): "))
```

☞ user থেকে error probability (p) নেওয়া হচ্ছে

- (p) = bit error হওয়ার probability
- range: 0 থেকে 1

★ Example:

- 0 → no error
 - 0.1 → 10% error
 - 0.5 → very noisy channel
-

◆ 3. Validity Check

```
if p < 0 or p > 1:
    print("Invalid probability!")
```

☞ probability যদি range-এর বাইরে যায়
 → □ error message দেখাবে

◆ 4. Edge Case Handle

```
else:
    if p == 0 or p == 1:
        H = 0
```

☞ দুইটা special case:

✓ $p = 0$

- কোনো error নেই
→ $\square$ uncertainty = 0

✓ $p = 1$

- সবসময় flip হয় (deterministic)
→ $\square$ আবারও uncertainty = 0

☞ তাই entropy = 0

◆ 5. Entropy Calculation

```
else:
    H = -p * math.log2(p) - (1 - p) * math.log2(1 - p)
```

☞ এটা হলো **Binary Entropy Function**

$$H(p) = -p \log_2(p) - (1 - p) \log_2(1 - p)$$

🔥 Meaning:

- (p) = error probability
- $(1-p)$ = correct transmission probability

☞ এটা বলে:

→ $\square$ channel-এর uncertainty কত

◆ 6. Channel Capacity

$$C = 1 - H$$

☞ এখানে capacity হিসাব করা হচ্ছে:

$$C = 1 - H(p)$$

🔥 Meaning:

- $H(p) \uparrow \rightarrow \text{noise} \uparrow \rightarrow \text{capacity} \downarrow$
 - $H(p) \downarrow \rightarrow \text{clean channel} \rightarrow \text{capacity} \uparrow$
-

◆ 7. Output Print

```
print("Entropy H(p) =", H)
print("Channel Capacity C =", C)
```

🔗 result দেখাচ্ছে:

- Entropy (uncertainty)
 - Channel capacity (maximum reliable transmission rate)
-

◆ 🔥 Full Concept (VERY IMPORTANT)

✓☐ Entropy ($H(p)$)

- channel-এর randomness measure করে
- noise কত বেশি তা বলে

✓☐ Capacity (C)

- কত information reliably পাঠানো যাবে
-

◆ Intuition

p	Channel condition	H(p)	C
0	perfect channel	0	1
0.5	maximum noise	1	0
1	deterministic flip	0	1

◆ Exam Ready Answer

☞ This program computes the binary entropy function $H(p)$ and channel capacity $C = 1 - H(p)$ for a Binary Symmetric Channel. It shows how increasing error probability increases uncertainty (entropy) and decreases the reliable transmission capacity of the channel.

🔥 One-line summary

☞ More noise $\rightarrow$ more entropy $\rightarrow$ less capacity

Code-5:

এটা খুব ভালো একটা **Shannon–Fano Coding implementation**। এখন আমি একদম **line-by-line** সহজভাবে বুঝিয়ে দিচ্ছি, যাতে তুমি algorithm + coding দুটোই clear বুঝো ☺

◆ 1. Function Definition

```
def shannon_fano(symbols, codes):
```

☞ এটা recursive function

→ □ Shannon–Fano coding করার জন্য

- `symbols` = (symbol, probability) list
 - `codes` = final binary code store করার dictionary
-

◆ 2. Base Condition

```
if len(symbols) <= 1:
    return
```

- ☞ যদি একটাই symbol থাকে
 - □ আর ভাগ করার দরকার নেই
 - □ recursion stop
-

◆ 3. Total Probability

```
total = sum([s[1] for s in symbols])
```

- ☞ সব symbol-এর probability যোগ করা হচ্ছে

- □ example:
 - A=0.4, B=0.2 → total = 1.0
-

◆ 4. Split Finding (Most Important 🔥)

```
acc = 0
split = 0
```

- ☞ accumulator এবং split point initialize
-

```
for i in range(len(symbols)):
    acc += symbols[i][1]
```

- ☞ একে একে probability যোগ করা হচ্ছে
-

```
if acc >= total / 2:
    split = i + 1
    break
```

- ☞ যখন cumulative sum half-এর কাছাকাছি পৌঁছে যায়
- □ তখনই list split করা হয়

★ Goal:

☞ দুইটা group প্রায় equal probability হবে

◆ 5. Splitting

```
left = symbols[:split]
right = symbols[split:]
```

☞ দুই ভাগ করা হলো:

- left group
 - right group
-

◆ 6. Code Assignment (0 and 1)

```
for s in left:
    codes[s[0]] += '0'
```

☞ left group-এর symbol-গুলোর code-এর শেষে '0' যোগ

```
for s in right:
    codes[s[0]] += '1'
```

☞ right group-এর symbol-গুলোর code-এর শেষে '1'

◆ 🔥 Key Idea

☞ Shannon–Fano works like:

- left $\rightarrow$ 0
- right $\rightarrow$ 1
- recursively repeat

→ □ তাই code gradually build হয়

◆ 7. Recursion Calls

```
shannon_fano(left, codes)
shannon_fano(right, codes)
```

☞ আবার same process repeat হবে

→ □ যতক্ষণ single symbol না হয়

◆ 8. Input Symbols

```
symbols = {
    'A': 0.4,
    'B': 0.2,
    'C': 0.2,
    'D': 0.1,
    'E': 0.1
}
```

☞ এগুলো probability distribution

→ □ A সবচেয়ে frequent

◆ 9. Sorting

```
sorted_symbols = sorted(symbols.items(), key=lambda x: x[1], reverse=True)
```

☞ probability অনুযায়ী descending order

★ কেন?

→ □ Shannon-Fano বড় probability আগে নেয়

◆ 10. Code Dictionary Initialize

```
codes = {symbol: "" for symbol in symbols}
```

☞ প্রতিটা symbol-এর জন্য empty code তৈরি

◆ 11. Run Algorithm

```
shannon_fano(sorted_symbols, codes)
```

☞ actual coding start

◆ 12. Output Print

```
print("Symbol\tProbability\tCode")
for s, p in sorted_symbols:
    print(f"{s}\t{p}\t\t{codes[s]}")
```

☞ final result show করবে:

- symbol
 - probability
 - generated binary code
-

🔥 Full Concept (Very Important)

✓ Shannon–Fano Idea:

☞ “Divide symbols into two groups with almost equal probability”

- □ Left = 0
 - □ Right = 1
 - □ recursively repeat
-

✓ Properties:

- lossless coding
- prefix code
- greedy splitting approach

◆ Exam Ready Answer

☞ **Shannon–Fano coding is a lossless source coding technique that assigns variable-length prefix codes to symbols based on their probabilities. It works by recursively dividing symbols into two groups with approximately equal total probability and assigning binary digits (0 and 1) to each group.**

🔥 One-line intuition

☞ **High probability → shorter code**
Low probability → longer code

Code-6:

এই কোডটা হলো **Huffman Coding implementation** — যা lossless optimal prefix coding-এর সবচেয়ে important example। এখন আমি একদম **line-by-line সহজভাবে বুঝিয়ে দিচ্ছি** 🙌

◆ 1. Library Import

```
import heapq
```

☞ `heapq` = priority queue (min-heap)

→ ☐ smallest probability আগে বের করার জন্য use হয়

◆ 2. Node Class (Tree বানানোর জন্য)

```
class Node:
```


☞ Huffman tree-এর প্রতিটা node represent করবে

```
def __init__(self, symbol, prob):
```

☞ প্রতিটা node-এর:

- symbol (A, B, C...)
- probability

```
self.symbol = symbol
self.prob = prob
self.left = None
self.right = None
```

☞ tree structure:

- left child
- right child

```
def __lt__(self, other):
    return self.prob < other.prob
```

☞ heapq compare করার জন্য

→ ☐ ছোট probability আগে যাবে

◆ 3. Code Generation Function

```
def generate_codes(node, code="", huffman_code={}):
```

☞ recursive function

→ ☐ tree traverse করে code generate করবে

```
if node is None:
    return
```

☞ base case (empty node)

```
if node.symbol is not None:
    huffman_code[node.symbol] = code
```

☞ যদি leaf node হয়

→ □ final binary code assign

```
generate_codes(node.left, code + "0", huffman_code)
generate_codes(node.right, code + "1", huffman_code)
```

☞ traversal rule:

- left → 0
- right → 1

```
return huffman_code
```

☞ final dictionary return

◆ 4. Input Symbols

```
symbols = {
    'A': 0.4,
    'B': 0.2,
    'C': 0.2,
    'D': 0.1,
    'E': 0.1
}
```

☞ probability distribution

→ □ A সবচেয়ে frequent

◆ 5. Priority Queue তৈরি

```
heap = []
```

☞ empty heap তৈরি

```
for symbol, prob in symbols.items():
    heapq.heappush(heap, Node(symbol, prob))
```

☞ প্রতিটা symbol heap-এ ঢুকানো হচ্ছে
 → □ probability অনুযায়ী order হবে

◆ 6. Huffman Tree Build

```
while len(heap) > 1:
```

☞ যতক্ষণ 1টা node না থাকে

```
left = heapq.heappop(heap)
right = heapq.heappop(heap)
```

☞ smallest 2 probability node বের করা

```
merged = Node(None, left.prob + right.prob)
```

☞ নতুন parent node তৈরি
 → □ probability যোগ করে

```
merged.left = left
merged.right = right
```

☞ tree connection তৈরি

```
heapq.heappush(heap, merged)
```

☞ আবার heap-এ push করা

◆ 7. Root Node

```
root = heap[0]
```

☞ final Huffman tree root

◆ 8. Code Generate

```
codes = generate_codes(root)
```

☞ পুরো tree traverse করে code তৈরি

◆ 9. Output Print

```
print("Symbol\tProbability\tCode")
```

☞ header print

```
for symbol in symbols:
    print(f"{symbol}\t{symbols[symbol]}\t\t{codes[symbol]}")
```

☞ final Huffman codes show

🔥 Core Concept (VERY IMPORTANT)

✓ Huffman idea:

☞ “Greedy approach: always combine two least probable symbols”

✓ Encoding rule:

- left = 0
 - right = 1
-

✓ Result:

- frequent symbol → short code

- rare symbol $\rightarrow$ long code
-

◆ Why optimal?

☞ Huffman coding gives:

- minimum average code length
 - best prefix code possible
-

◆ Exam Ready Answer

☞ **Huffman coding is an optimal lossless source coding technique that builds a binary tree using a greedy algorithm by repeatedly combining the two least probable symbols. It assigns shorter codes to more frequent symbols and longer codes to less frequent ones, resulting in minimum average code length.**

🔥 One-line intuition

☞ **Less probability $\rightarrow$ longer code**
More probability $\rightarrow$ shorter code